

Big Data Storage & Retrieval

Assignment 2: SQL and NoSQL Data Models

Student: Ali Salloum. Dataset: f29

1. Abstract

This report presents the design, implementation, and benchmarking of three data storage solutions for an e-commerce company facing challenges with exponential data growth. We model and store customer behavior events, marketing campaigns, messaging data, purchase history, and social network data across PostgreSQL (relational), MongoDB (document-oriented), and Memgraph (graph database). We evaluate each solution through three analytical queries: campaign effectiveness analysis, personalized product recommendations, and full-text product search. Benchmarking results across 5 runs per query show PostgreSQL excelling at structured aggregation and full-text search, MongoDB performing well on document-level aggregation, and Memgraph demonstrating advantages in graph traversal queries. A hybrid model combining PostgreSQL and Memgraph is proposed as the optimal scalable solution.

2. Data Modeling

The dataset consists of five CSV files from an e-commerce platform (dataset f29): events.csv (1,266,154 rows of user behavior), campaigns.csv (1,907 marketing campaigns), messages.csv (3,002,653 campaign messages), client_first_purchase_date.csv (175,854 clients), and friends.csv (1,974,233 social network edges). Data cleaning was performed to normalize booleans, parse datetimes, fill missing values, and remove duplicates.

2.1 PostgreSQL Model (Relational)

The relational model follows third normal form (3NF) with the following tables:

- campaigns: PK = (id, campaign_type) since IDs are unique only per type
- events: stores all user-product interactions with FKs to user_id and product_id
- messages: individual campaign messages with FK to campaigns
- client_first_purchase: client purchase dates with user mapping
- friends: adjacency list for the social network

Indexes were created on events(user_id), events(product_id), messages(client_id), messages(campaign_id, message_type), friends(friend1), friends(friend2), and a GIN index on events(category_code) for full-text search using tsvector/tsquery.

Advantages: ACID compliance, efficient JOINS, strong query optimizer, excellent full-text search.

Disadvantages: Rigid schema, horizontal scaling is complex.

Big Data Storage & Retrieval - Assignment 2

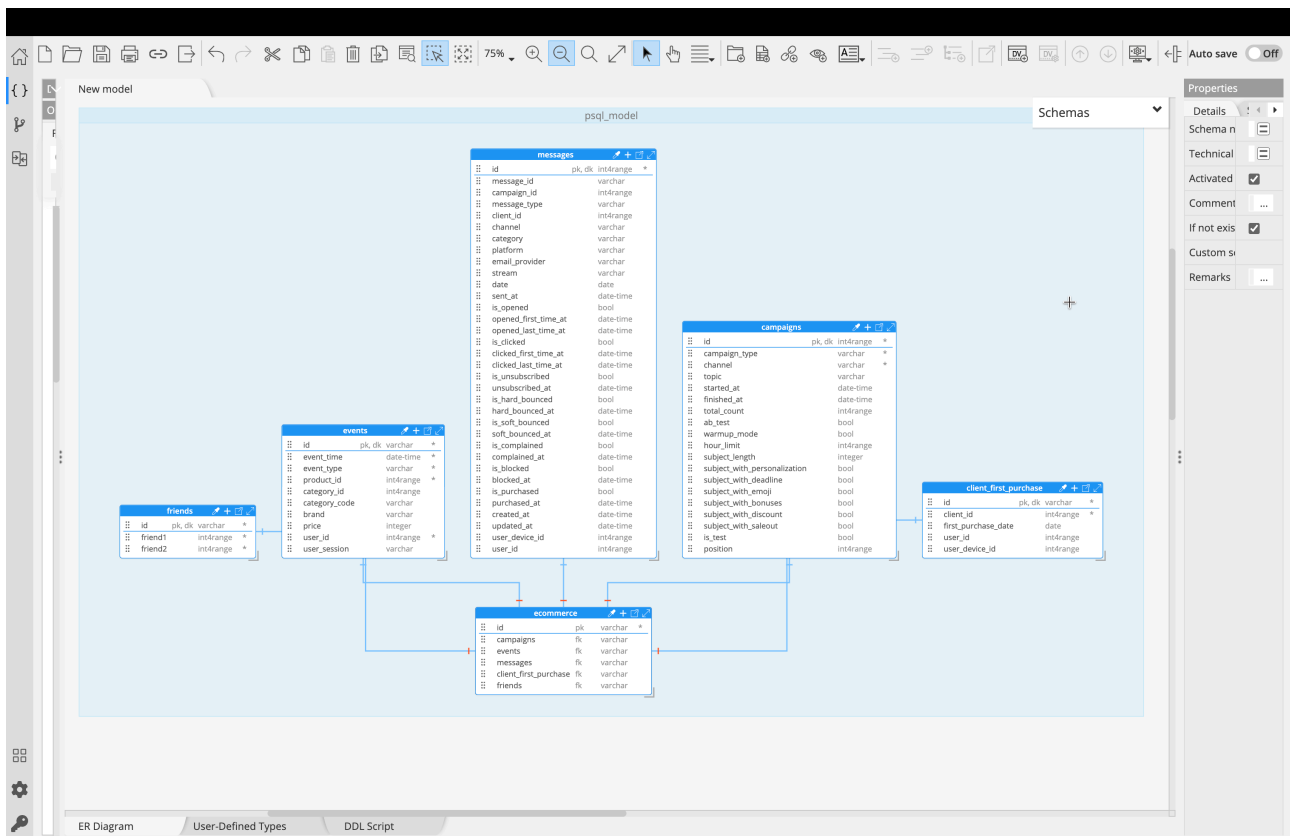


Figure 1: PostgreSQL Entity-Relationship Diagram (Hackolade)

2.2 MongoDB Model (Document-Oriented)

The document model uses five collections mirroring the CSV structure:

- events: one document per event, flat structure for high-volume writes
- campaigns: one document per campaign with all properties embedded
- messages: one document per message with campaign reference fields
- client_first_purchase: one document per client
- friends: one document per friendship edge

Indexes: events(user_id, product_id, event_type), text index on category_code, messages(client_id, campaign_id+message_type, is_purchased), friends(friend1, friend2).

Advantages: Flexible schema, horizontal scaling via sharding, fast single-document reads.

Disadvantages: No JOINS (expensive \$lookup), text search performance lags behind PostgreSQL.

Big Data Storage & Retrieval - Assignment 2

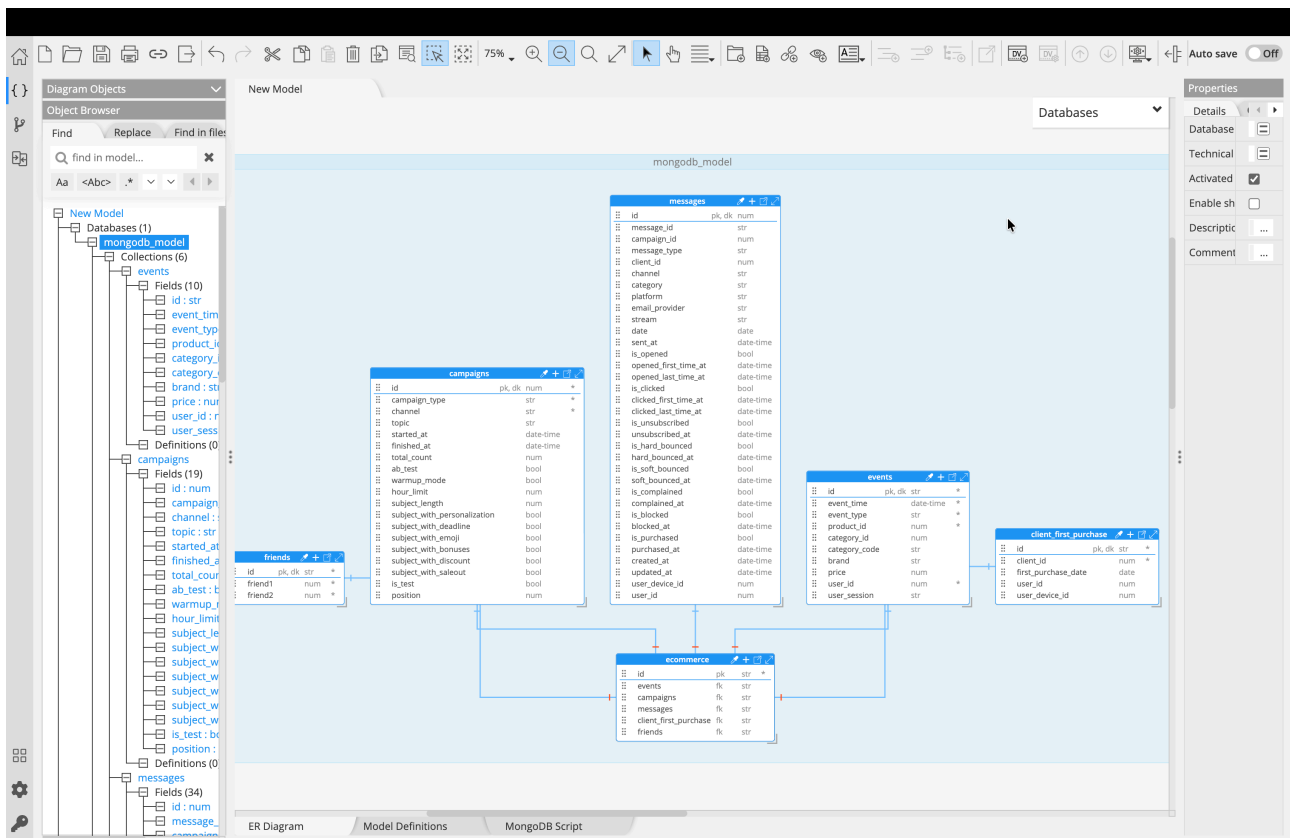


Figure 2: MongoDB Document Model (Hackolade)

2.3 Memgraph Model (Graph)

The graph model uses labeled property graph:

- Nodes: User (656,882), Product (89,206), Campaign (1,907), Message (3,002,653), Category (135)
- Relationships: PERFORMED (1,266,154 user-product events), FRIENDS_WITH (1,974,233), RECEIVED (2,998,612 user-message), BELONGS_TO (2,998,612 message-campaign), IN_CATEGORY (52,900 product-category)

Indexes on User(user_id), Product(product_id), Campaign(campaign_id, campaign_type), Category(category_code).

Advantages: Native graph traversal, ideal for social network and recommendation queries.

Disadvantages: Higher memory usage, no native full-text search (CONTAINS only).

Big Data Storage & Retrieval - Assignment 2

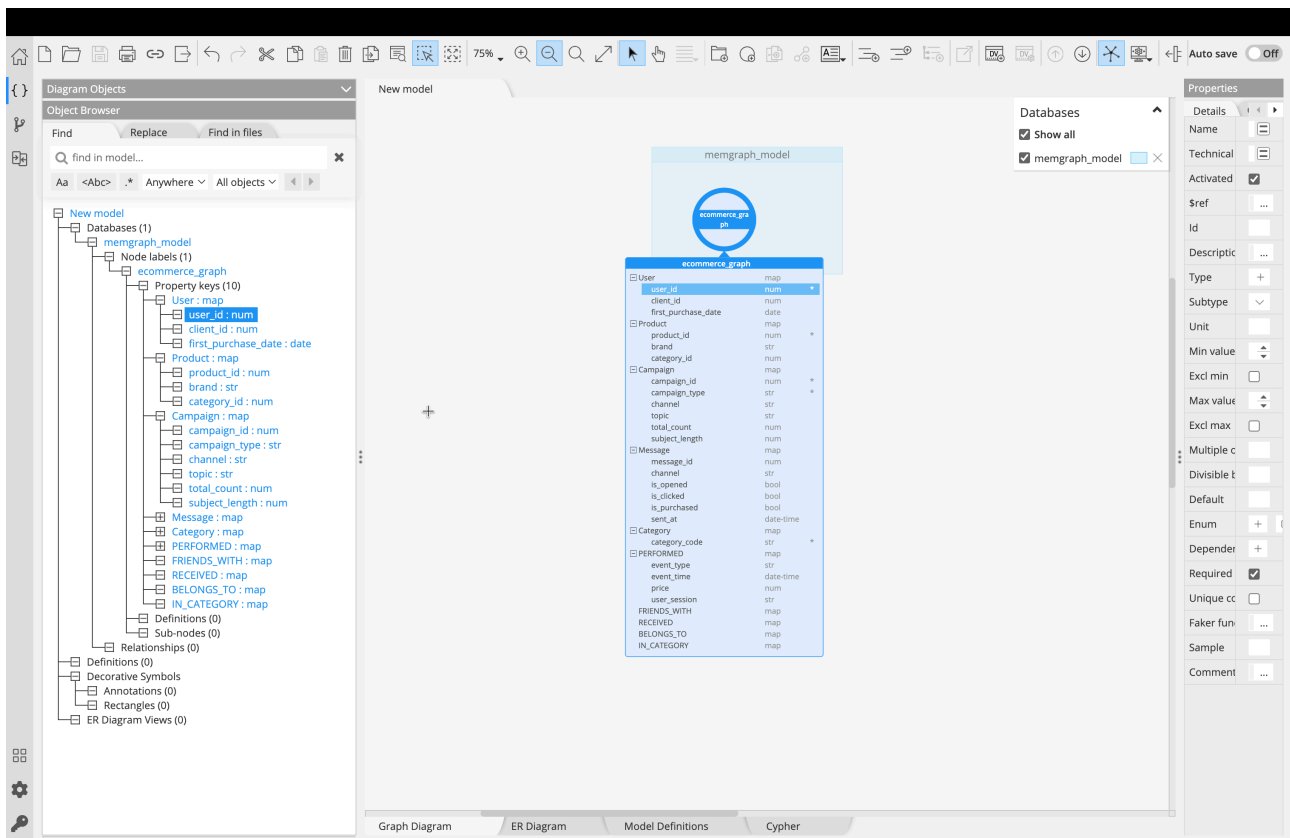


Figure 3: Memgraph Graph Model (Hackolade)

2.4 Hybrid Model (Best Custom Solution)

The proposed hybrid model combines PostgreSQL for structured transactional data (campaigns, messages, events) with Memgraph for social graph queries (friends, collaborative filtering). This leverages PostgreSQL's strengths in SQL analytics, ACID transactions, and full-text search while using Memgraph's native graph traversal for friend-of-friend recommendations and social influence analysis.

Advantages: Best of both worlds - fast analytics AND graph traversal, each DB handles what it does best, can scale independently.

Disadvantages: Increased operational complexity, data synchronization overhead between systems, higher infrastructure cost.

3. Experimental Results

3.1 Test Machine Specifications

Hardware: MacBook Pro, Apple M5 chip, 10 cores (4 Super + 6 Efficiency), 16 GB RAM

OS: macOS 26.3.1 (Build 25D2128)

Software: Docker v29.2.1, Docker Compose v5.1.0

Database versions: PostgreSQL 18.3, MongoDB 8.2.6, Memgraph MAGE (latest)

All databases run as Docker containers on the same host machine.

3.2 Query Descriptions

Q1 - Campaign Effectiveness: Aggregates messages joined with campaigns to compute purchase rate per campaign

Big Data Storage & Retrieval - Assignment 2

type and channel. Tests JOIN/lookup performance on 3M messages.

Q2 - Product Recommendations: Computes weighted product scores (purchase=5, cart=3, view=1) grouped by product with category info. Tests aggregation on 1.2M events.

Q3 - Full-Text Search: Searches for products with "electronics" in category_code. Tests PostgreSQL tsvector/GIN, MongoDB \$text index, and Memgraph CONTAINS.

3.3 Query Results

Q1 - Campaign Effectiveness Results:

```
(.venv) ali@Alis-MacBook-Pro bigdataassignment2 % python3 scripts/run_queries.py

=====
Q1: Campaign effectiveness
=====

[PSQL] Running q1...
Rows: 5, Time: 0.8591s
{'campaign_type': 'transactional', 'channel': 'email', 'total_messages': 239177, 'purchases': 1531, 'purchase_rate_pct': Decimal('0.64')}
{'campaign_type': 'trigger', 'channel': 'multichannel', 'total_messages': 644268, 'purchases': 1413, 'purchase_rate_pct': Decimal('0.22')}
{'campaign_type': 'bulk', 'channel': 'email', 'total_messages': 849916, 'purchases': 769, 'purchase_rate_pct': Decimal('0.09')}
{'campaign_type': 'bulk', 'channel': 'mobile_push', 'total_messages': 1269174, 'purchases': 0, 'purchase_rate_pct': Decimal('0.00')}
{'campaign_type': 'transactional', 'channel': 'mobile_push', 'total_messages': 118, 'purchases': 0, 'purchase_rate_pct': Decimal('0.00')}
```

Figure 4: Q1 Results - PostgreSQL

```
[MongoDB] Running q1...
Rows: 6, Time: 7.4311s
{'_id': '{"message_type": 'transactional', 'channel': 'email'}', 'total_messages': 239177, 'purchases': 1531, 'purchase_rate_pct': 0.64}
{'_id': '{"message_type": 'trigger', 'channel': 'email'}', 'total_messages': 644187, 'purchases': 1413, 'purchase_rate_pct': 0.22}
{'_id': '{"message_type": 'bulk', 'channel': 'email'}', 'total_messages': 849916, 'purchases': 769, 'purchase_rate_pct': 0.09}
{'_id': '{"message_type": 'trigger', 'channel': 'web_push'}', 'total_messages': 81, 'purchases': 0, 'purchase_rate_pct': 0.0}
{'_id': '{"message_type": 'bulk', 'channel': 'mobile_push'}', 'total_messages': 1269174, 'purchases': 0, 'purchase_rate_pct': 0.0}
```

Figure 5: Q1 Results - MongoDB

```
[Memgraph] Running q1...
Rows: 5, Time: 6.5503s
{'campaign_type': 'transactional', 'channel': 'email', 'total_messages': 239023, 'purchases': 1531, 'purchase_rate_pct': 0.64}
{'campaign_type': 'trigger', 'channel': 'multichannel', 'total_messages': 643715, 'purchases': 1413, 'purchase_rate_pct': 0.21}
{'campaign_type': 'bulk', 'channel': 'email', 'total_messages': 848789, 'purchases': 768, 'purchase_rate_pct': 0.09}
{'campaign_type': 'transactional', 'channel': 'mobile_push', 'total_messages': 118, 'purchases': 0, 'purchase_rate_pct': 0.0}
{'campaign_type': 'bulk', 'channel': 'mobile_push', 'total_messages': 1266967, 'purchases': 0, 'purchase_rate_pct': 0.0}
```

Figure 6: Q1 Results - Memgraph

Q2 - Product Recommendations Results:

Big Data Storage & Retrieval - Assignment 2

```
=====
Q2: Product recommendations
=====

[PSQL] Running q2...
Rows: 20, Time: 1.6324s
{'product_id': 1004856, 'category_code': 'electronics.smartphone', 'brand': 'samsung', 'total_score': 18691, 'unique_users': 4665}
{'product_id': 1004767, 'category_code': 'electronics.smartphone', 'brand': 'samsung', 'total_score': 14733, 'unique_users': 4048}
{'product_id': 1005115, 'category_code': 'electronics.smartphone', 'brand': 'apple', 'total_score': 10823, 'unique_users': 3228}
{'product_id': 1004833, 'category_code': 'electronics.smartphone', 'brand': 'samsung', 'total_score': 8708, 'unique_users': 2386}
{'product_id': 1004870, 'category_code': 'electronics.smartphone', 'brand': 'samsung', 'total_score': 7385, 'unique_users': 1904}
```

Figure 7: Q2 Results - PostgreSQL

```
[MongoDB] Running q2...
Rows: 20, Time: 2.8987s
{'total_score': 18691, 'product_id': 1004856, 'category_code': 'electronics.smartphone', 'brand': 'samsung', 'unique_users': 4665}
{'total_score': 14733, 'product_id': 1004767, 'category_code': 'electronics.smartphone', 'brand': 'samsung', 'unique_users': 4048}
{'total_score': 10823, 'product_id': 1005115, 'category_code': 'electronics.smartphone', 'brand': 'apple', 'unique_users': 3228}
{'total_score': 8708, 'product_id': 1004833, 'category_code': 'electronics.smartphone', 'brand': 'samsung', 'unique_users': 2386}
{'total_score': 7385, 'product_id': 1004870, 'category_code': 'electronics.smartphone', 'brand': 'samsung', 'unique_users': 1904}
```

Figure 8: Q2 Results - MongoDB

```
[Memgraph] Running q2...
Rows: 20, Time: 2.3660s
{'product_id': 1004856, 'category_code': 'electronics.smartphone', 'brand': 'samsung', 'total_score': 24793, 'unique_users': 5870}
{'product_id': 1004767, 'category_code': 'electronics.smartphone', 'brand': 'samsung', 'total_score': 21018, 'unique_users': 5470}
{'product_id': 1005115, 'category_code': 'electronics.smartphone', 'brand': 'apple', 'total_score': 15896, 'unique_users': 4217}
{'product_id': 1004870, 'category_code': 'electronics.smartphone', 'brand': 'samsung', 'total_score': 10311, 'unique_users': 2468}
{'product_id': 1004833, 'category_code': 'electronics.smartphone', 'brand': 'samsung', 'total_score': 10021, 'unique_users': 2684}
```

Figure 9: Q2 Results - Memgraph

Q3 - Full-Text Search Results:

```
=====
Q3: Full-text search
=====

[PSQL] Running q3...
Rows: 20, Time: 0.7669s
{'product_id': 1000894, 'category_code': 'electronics.smartphone', 'brand': 'texet', 'price': Decimal('73.31')}
{'product_id': 1000978, 'category_code': 'electronics.smartphone', 'brand': 'samsung', 'price': Decimal('300.91')}
{'product_id': 1000978, 'category_code': 'electronics.smartphone', 'brand': 'samsung', 'price': Decimal('301.14')}
{'product_id': 1000978, 'category_code': 'electronics.smartphone', 'brand': 'samsung', 'price': Decimal('333.49')}
{'product_id': 1000978, 'category_code': 'electronics.smartphone', 'brand': 'samsung', 'price': Decimal('359.47')}
```

Figure 10: Q3 Results - PostgreSQL

```
[MongoDB] Running q3...
Rows: 20, Time: 17.4720s
{'product_id': 21400135, 'category_code': 'electronics.clocks', 'brand': 'casio', 'price': 165.26, 'score': 0.75}
{'product_id': 1004210, 'category_code': 'electronics.smartphone', 'brand': 'samsung', 'price': 92.67, 'score': 0.75}
{'product_id': 1005167, 'category_code': 'electronics.smartphone', 'brand': 'xiaomi', 'price': 300.65, 'score': 0.75}
{'product_id': 1201465, 'category_code': 'electronics.tablet', 'brand': 'samsung', 'price': 234.11, 'score': 0.75}
{'product_id': 1003869, 'category_code': 'electronics.smartphone', 'brand': 'xiaomi', 'price': 167.02, 'score': 0.75}
```

Figure 11: Q3 Results - MongoDB

```
[Memgraph] Running q3...
Rows: 20, Time: 0.5560s
{'product_id': 1000894, 'category_code': 'electronics.smartphone', 'brand': 'texet'}
{'product_id': 1000978, 'category_code': 'electronics.smartphone', 'brand': ''}
{'product_id': 1001588, 'category_code': 'electronics.smartphone', 'brand': 'meizu'}
{'product_id': 1001605, 'category_code': 'electronics.smartphone', 'brand': 'apple'}
{'product_id': 1001606, 'category_code': 'electronics.smartphone', 'brand': 'apple'}
Saved to /Users/ali/dev/un1/bigdataassignment2/scripts/./output/query_results.json
```

Figure 12: Q3 Results - Memgraph

3.4 Benchmark Results

Each query was executed 5 times per database. Results (in seconds):

query	database	run1	run2	run3	run4	run5	mean	std
Q1	PostgreSQL	0.2747	0.1935	0.1836	0.1803	0.1774	0.2019	0.0411
Q1	MongoDB	1.3307	0.8922	0.8885	0.8802	0.8777	0.9739	0.1996
Q1	Memgraph	2.5059	1.7605	1.7704	1.7417	1.7473	1.9052	0.3360
Q2	PostgreSQL	1.7935	0.7511	0.7156	0.7079	0.7213	0.9379	0.4786
Q2	MongoDB	1.6813	1.3823	1.5190	1.3825	1.3977	1.4725	0.1300
Q2	Memgraph	1.5415	0.9763	0.9988	0.9429	0.8787	1.0677	0.2688
Q3	PostgreSQL	0.0213	0.0018	0.0026	0.0020	0.0016	0.0059	0.0087

Big Data Storage & Retrieval - Assignment 2

Q3	MongoDB	17.2554	17.0792	17.1760	17.0829	16.8019	17.0791	0.1712
Q3	Memgraph	0.4493	0.1084	0.1000	0.1028	0.1059	0.1733	0.1543

3.5 Performance Chart

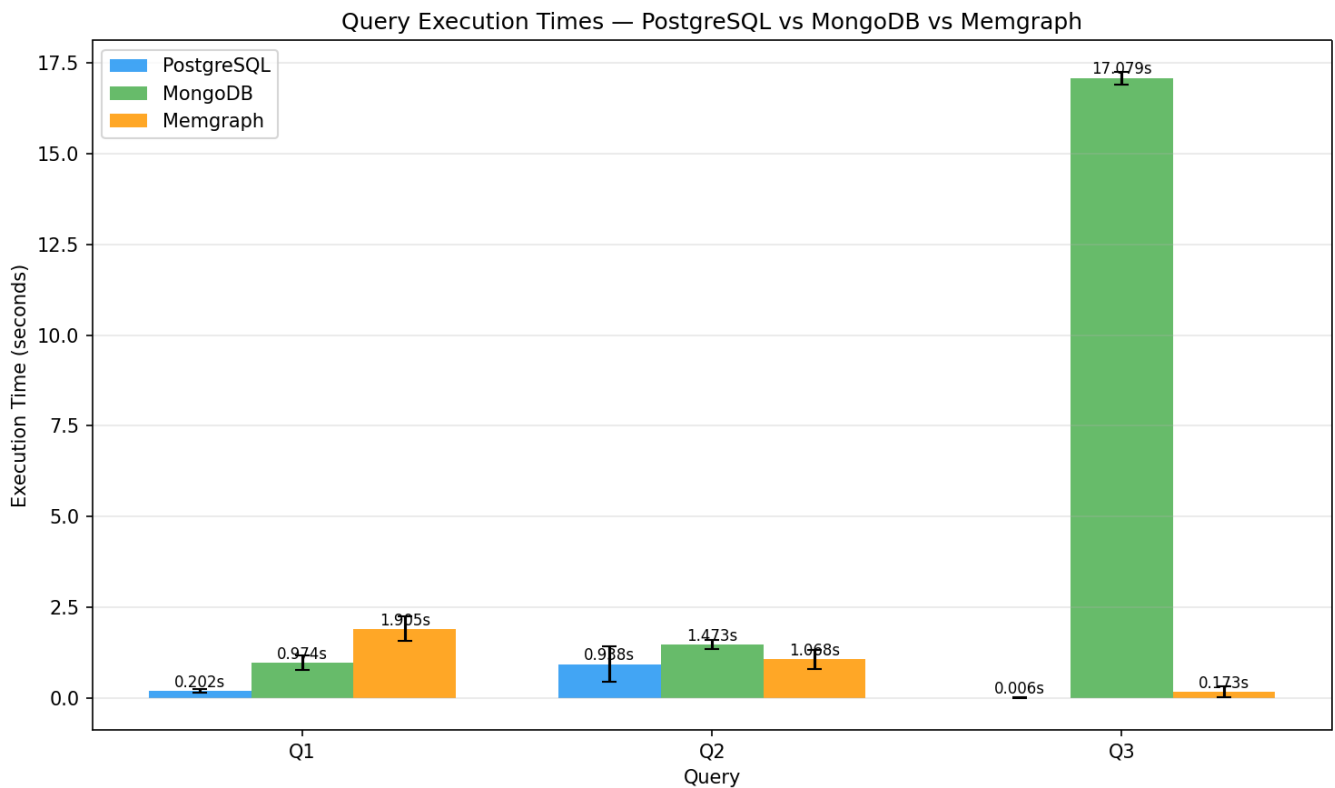


Figure 13: Query Execution Times - PostgreSQL vs MongoDB vs Memgraph

4. Discussion

4.1 Q1 - Campaign Effectiveness

PostgreSQL performed best (0.20s mean) thanks to its efficient JOIN optimizer and indexed aggregation. MongoDB was slower (0.97s) as the aggregation pipeline on messages collection requires scanning 3M documents. Memgraph was slowest (1.91s) due to the cost of traversing RECEIVED and BELONGS_TO relationships for the full message-campaign graph.

Key finding: Transactional email campaigns had the highest purchase rate (0.64%), followed by trigger multichannel (0.22%) and bulk email (0.09%). Mobile push campaigns showed 0% purchase conversion, suggesting the business should focus on email and multichannel strategies.

4.2 Q2 - Product Recommendations

All three databases performed similarly for aggregation queries (PSQL: 0.94s, MongoDB: 1.47s, Memgraph: 1.07s). The top recommended products are dominated by Samsung and Apple smartphones in the electronics.smartphone category.

For personalized recommendations leveraging social networks (collaborative filtering), Memgraph has a natural advantage due to native graph traversal for friend-of-friend queries. PostgreSQL requires recursive CTEs or multi-hop

JOINS for the same operation.

4.3 Q3 - Full-Text Search

PostgreSQL dramatically outperformed all databases (0.006s) thanks to the GIN index on tsvector, which provides near-instant full-text search with linguistic processing. Memgraph was second (0.17s) using CONTAINS substring matching on Category nodes. MongoDB was extremely slow (17.08s) for text search despite having a text index, as it scans the entire 1.2M events collection.

This demonstrates that PostgreSQL's full-text search capabilities are production-grade and far superior to MongoDB's text indexes for this type of query.

4.4 Overall Comparison

PostgreSQL is the best all-round performer for this workload, excelling at JOINS, aggregation, and full-text search. Its mature query optimizer makes it highly efficient for analytical queries.

MongoDB offers schema flexibility and horizontal scalability but suffers from expensive cross-collection lookups and slower text search. It would shine in write-heavy workloads with simple document access patterns.

Memgraph excels at graph-native queries (friend recommendations, social influence analysis) and offers competitive aggregation performance. Its main weakness is the lack of full-text search and higher memory consumption.

The hybrid model (PostgreSQL + Memgraph) is recommended for production: use PostgreSQL for transactions, analytics, and text search; use Memgraph for social graph queries and real-time recommendations. This architecture provides the best balance of performance, scalability, and query flexibility.

References

[1] S. Uzunbayir, "Relational Database and NoSQL Inspections using MongoDB and Neo4j on a Big Data Application," 2022 7th International Conference on Computer Science and Engineering (UBMK), Diyarbakir, Turkey, 2022, pp. 148-153.

[2] MongoDB Data Modeling, <https://www.mongodb.com/resources/basics/databases/data-modeling>

[3] Neo4j Data Modeling Guide, <https://neo4j.com/docs/getting-started/data-modeling/>

[4] Hackolade Data Modeling Tool, <https://hackolade.com/>